

Dynamic Programming

Analysis and Design of Algorithms



Dynamic Programming (DP)

is an algorithmic technique used to solve optimization problems by breaking them into overlapping subproblems and storing (memoizing) the results of already-solved subproblems to avoid redundant computation.



Key Characteristics

Optimal Substructure

The optimal solution of the problem can be constructed from optimal solutions of its subproblems.

Overlapping Subproblems

Sub-problems repeat many times (unlike Divide-and-Conquer, where subproblems are independent).

Memoization / Tabulation

DP uses storage to save time.



DP Approaches

(A) Top-Down (Memoization)

Write the solution recursively.

Store results of each subproblem in a dictionary/table.

Avoid recomputation.

(B) Bottom-Up (Tabulation)

Build a table iteratively from base cases.

No recursion required.

Often faster and more memory-efficient.



Example 1: 0/1 Knapsack Problem

- **Problem Definition**

Given:

A set of n items

Each item has:

- weight $w[i]$
- value $v[i]$

A knapsack with maximum capacity W

Goal:

Maximize the total value of selected items **without exceeding the capacity**.

Each item can be taken **0 or 1 time** → *0/1 Knapsack*.



Example 1: 0/1 Knapsack Problem

Let

$dp[i][w]$ = maximum value achievable using the **first i items** with capacity **w** .



Example 1: 0/1 Knapsack Problem

Item	Weight	Value
1	1	1
2	3	4
3	4	5
4	5	7

Capacity $W = 7$

You can walk students through filling the table row by row.

Complexity

Time: $O(nW)$

Space: $O(nW)$ (can be reduced to $O(W)$ in optimized version)



Example 2: Longest Common Subsequence (LCS)

- **Problem Definition**

Given two sequences:

- X of length m
- Y of length n
- Find the **longest subsequence** common to both strings.
A subsequence keeps **order** but not necessarily contiguity.

Goal:

X = **A B C D G H**

Y = **A E D F H R**

LCS = **A D H**



Example 2: Longest Common Subsequence (LCS)

- Let
 $dp[i][j]$ = LCS length of prefixes $X[1..i]$ and $Y[1..j]$.



Example 2: Longest Common Subsequence (LCS)

- **DP Table Example**

X = **A**BCBDAB

Y = **B**DCABA

- Students fill a 2D table of size $(\text{len}(X)+1) \times (\text{len}(Y)+1)$.



Example 2: Longest Common Subsequence (LCS)

- **Reconstructing the LCS**

Start from bottom-right cell $dp[m][n]$:

If characters match $\rightarrow$ include that character and move diagonally

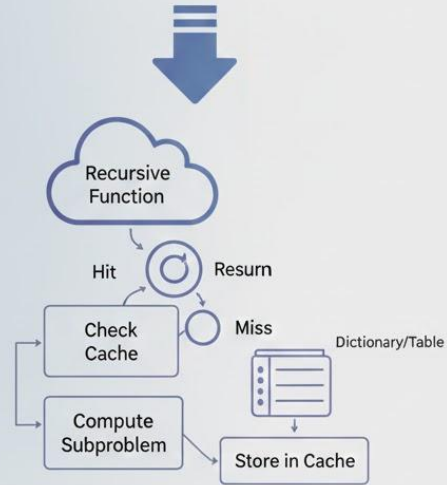
Else move to the direction of the larger value (up or left)

- **Complexity**

- Time: $O(mn)$
- Space: $O(mn)$ (can be optimized)

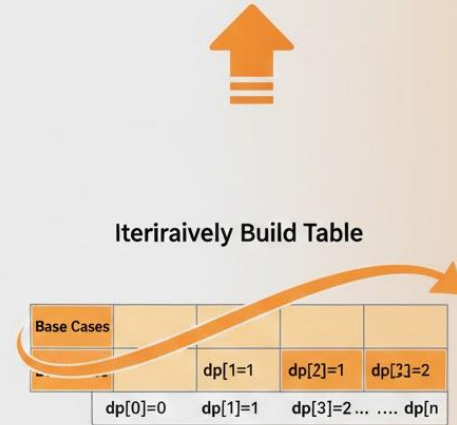


(A) TOP-DOWN (MEMOIZATION)



-  Write solution recursively
-  Store results...
avoid recompation
-  Avoid recomuization

(B) BOTTOM-UP (TABULATION)



-  Build a table iteratively...
-  No recursion required
-  Often faster...
memory-efficient

